ANÁLISIS DEL COSTE COMPUTACIONAL

Carlos Pérez

Departamento de Lenguajes y Sistemas Informáticos
Universidad de Alicante
Curso 2026-2027



Esta obra se publica con una licencia BY-NC-SA Creative Commons License

Index

1. Objetivos
2. Noción de coste computacional
3. Cotas de complejidad
4. Análisis asintótico
5. Cálculo de complejidades

Index

1. Objetivos
2. Noción de coste computacional
3. Cotas de complejidad
4. Análisis asintótico
5. Cálculo de complejidades

Objetivos

- Distinguir los conceptos de **eficiencia en tiempo y espacio**.
- Entender y saber aplicar **criterios asintóticos** a los conceptos de eficiencia.
- Saber calcular el **coste temporal o espacial** de un algoritmo.
- Saber **comparar** distintas soluciones algorítmicas de un mismo problema desde el punto de vista de la eficiencia.

Index

1. Objetivos
2. Noción de coste computacional
3. Cotas de complejidad
4. Análisis asintótico
5. Cálculo de complejidades

Introducción

Estamos interesados en medir de alguna forma el **coste de los recursos** que consume un algoritmo en su ejecución **(complejidad/coste computacional)**.

El uso eficiente de los recursos se mide en términos de 2 parámetros:

- **ESPACIO** (memoria que consume $\rightarrow$ **coste espacial**)
- **TIEMPO** (lo que tarda en ejecutarse $\rightarrow$ **coste temporal**)

Objetivo: minimizar el coste temporal y espacial (compromiso)

Con estas medidas podemos hacer ...

- **Valoraciones:** A es prohibitivo, A es rápido
- **Comparaciones:** A es mejor que B, A es el mejor

Nos centraremos en el coste temporal (crítico), aunque hablaremos también del coste espacial.

Introducción

Cuando hacemos las valoraciones o comparaciones, estamos teniendo en cuenta que:

- la elección del algoritmo no debería afectar a su solución
- **excepción:** la elección puede afectar cuando la solución correcta es inmanejable (algoritmos aproximados $\Rightarrow$ solución no óptima pero aceptable)
- hay que ser conscientes de que existen **problemas intratables** (coste muy elevado)

Tiempo de ejecución de un algoritmo

El tiempo de ejecución de un algoritmo dependerá de varios factores:

Externos

- la máquina en la que se va a ejecutar
- el compilador
- los datos de entrada (naturaleza y número)
- la experiencia del programador

Internos

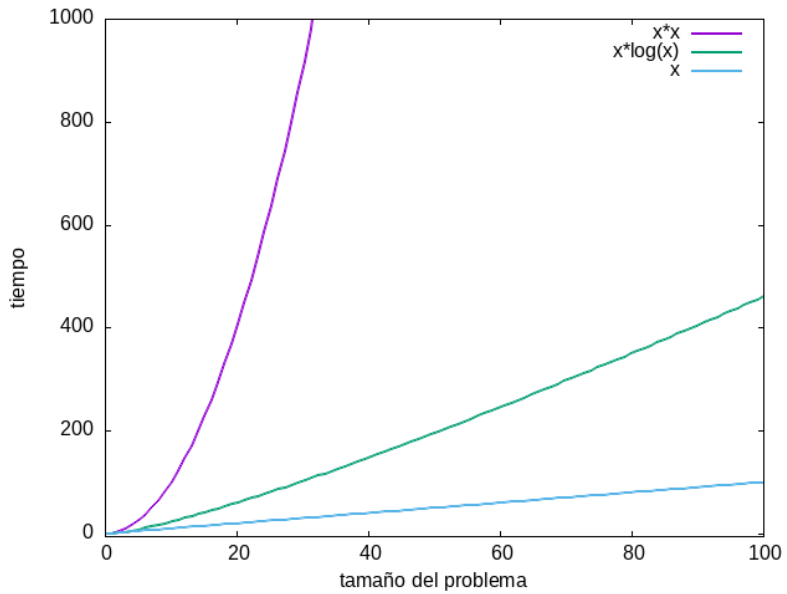
- complejidad intrínseca del algoritmo (número de instrucciones)

Tiempo de ejecución de un algoritmo

Existen 2 estudios posibles sobre el tiempo:

1) Estudio analítico (a priori): se obtiene una **función** que representa el tiempo de ejecución para distintos valores de entrada

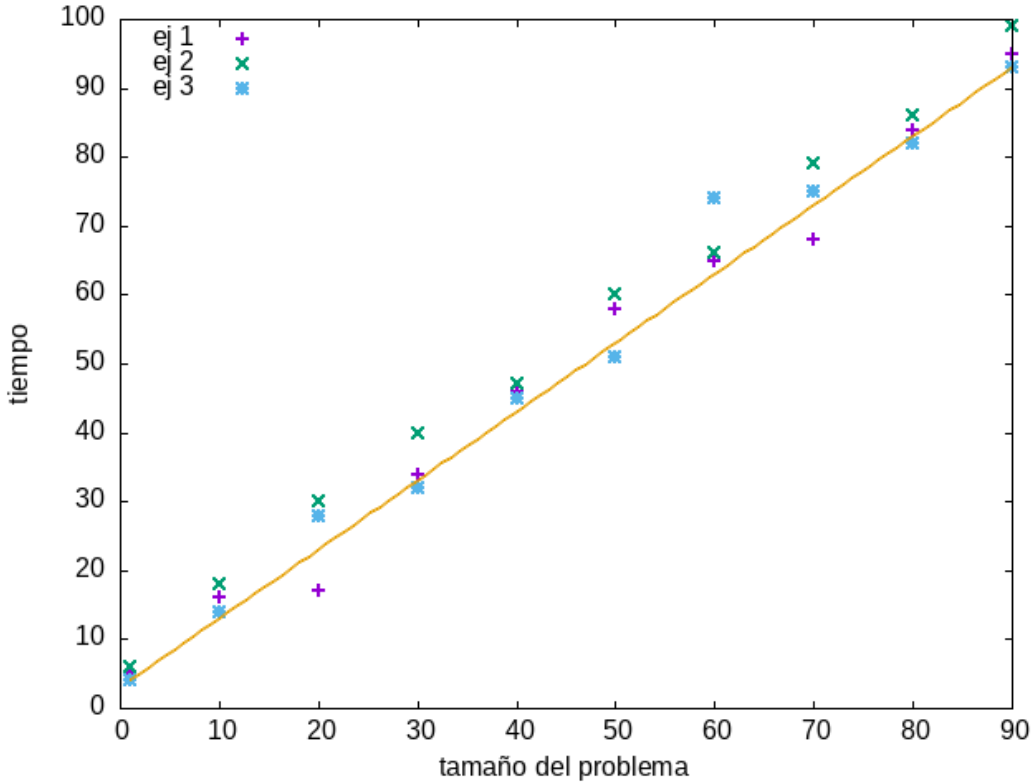
- ▲ el resultado depende solo de factores internos
- ▲ no es necesario implementar y ejecutar el algoritmo → lenguaje de alto nivel (pseudocódigo)
- ▼ no obtiene una medida real del comportamiento del algoritmo en el entorno de aplicación



Tiempo de ejecución de un algoritmo

2) Estudio empírico (a posteriori): consistente en **medir** el tiempo de ejecución del algoritmo para unos valores de entrada dados

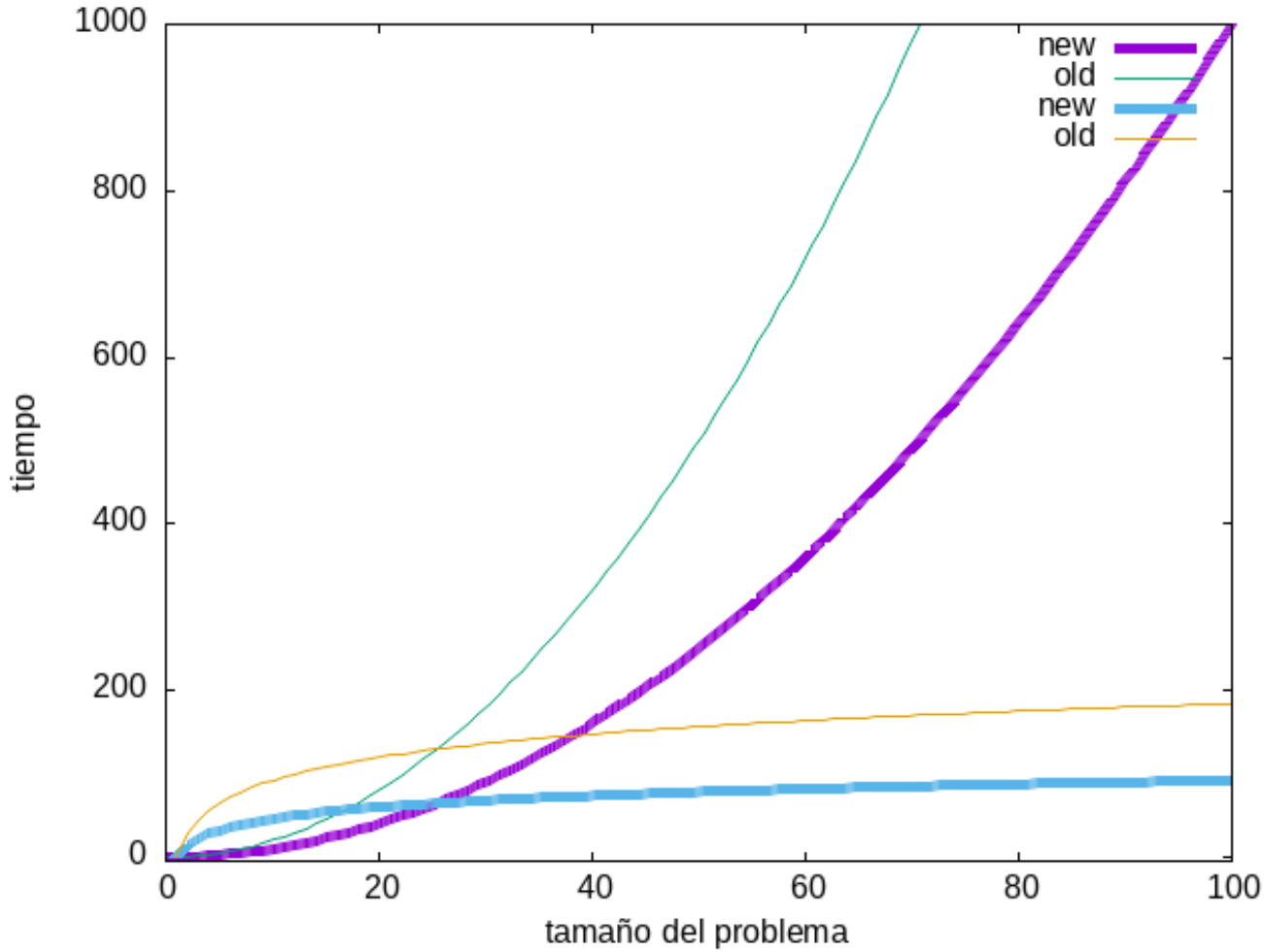
- ▲ se obtiene una medida real del comportamiento del algoritmo en el entorno de aplicación
- ▼ el resultado depende de factores internos y externos



Sin embargo, ¡ambas medidas son importantes!

Tiempo de ejecución de un algoritmo

¿Cómo afecta al coste de un algoritmo el hardware?



Tiempo de ejecución de un algoritmo

- n: **tamaño de la entrada** (número de elementos sobre los que se va a ejecutar el algoritmo).
- $T(n)$: **tiempo de ejecución** de un algoritmo para una entrada de tamaño n.

$T(n)$ está relacionado con el número de **operaciones elementales**.

Operaciones elementales (OE): operaciones que realiza el ordenador en tiempo, acotado por una constante:

- operaciones aritméticas básicas ($+$ - / * etc)
- asignaciones a variables
- saltos (llamadas a funciones o retorno desde ellos ...)
- comparaciones lógicas (and, or)
- acceso a estructuras indexadas básicas (vectores o matrices)

Reglas generales para el cálculo del número de OE

Secuencia:

1) n = 1;
2) x = x+3;
3) y = n * x;

→ $T = T1 + T2 + T3 = 1 + 2 + 2$

Selección:

Si (C)
entonces S1;
sino S2;

→ $T = T(C) + max(T(S1), T(S2))$

Bucles:

Mientras (C)
hacer S;

→ $T = T(C) + n^o\ iteraciones*(T(C) + T(S))$

Recursión:

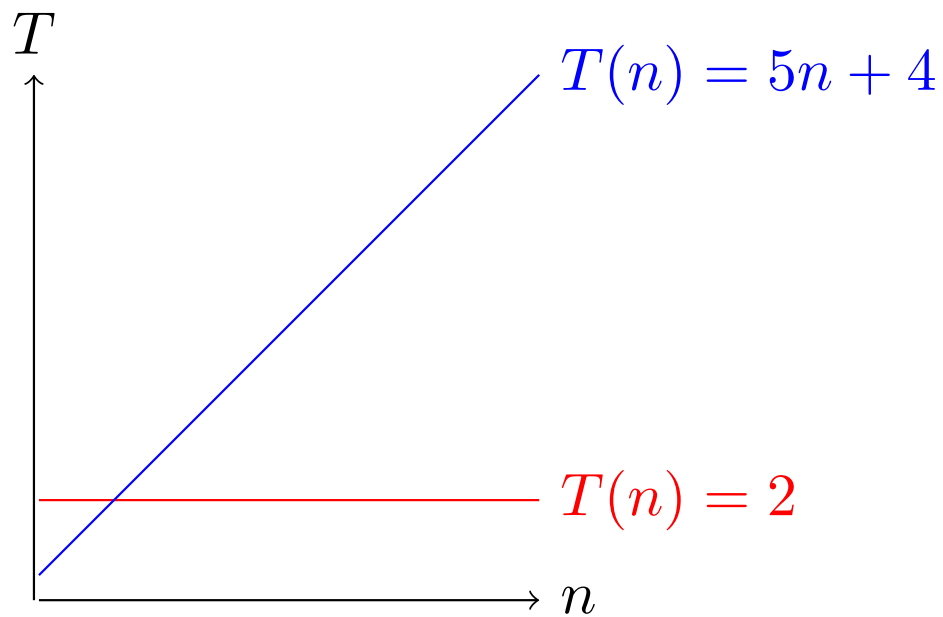
ecuaciones
de recurrencia

→ $T(n) = x + T(n-1)$
$T(n) = 1 + 2T(n/2)$

Tiempo de ejecución de un algoritmo

$T(n)$ se determina con una **función** que mide el **número de OE** que realiza el algoritmo. La expresión de esta función debe ser válida para cualquier entrada.

n se refiere, p.e., a la dimensión de un vector, tamaño de una matriz, tamaño de un lista, etc.



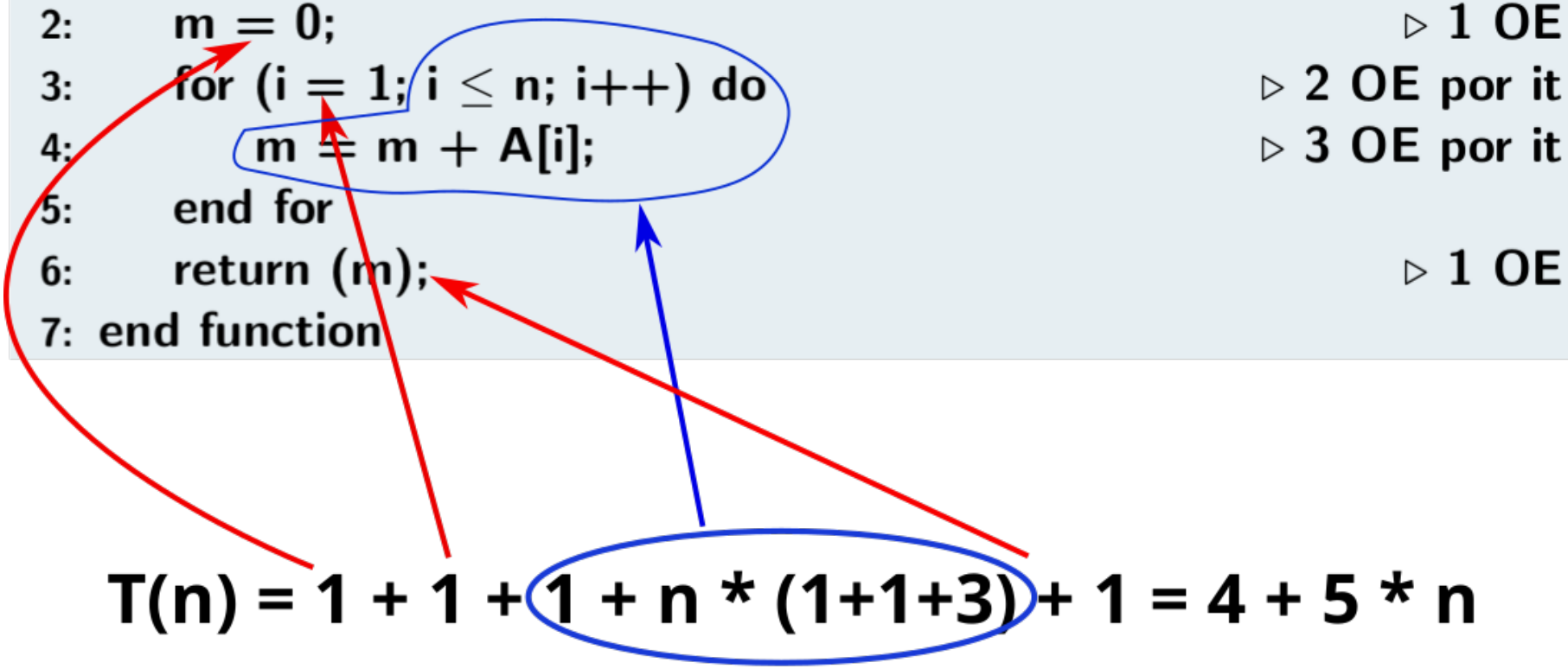
Ejemplo: suma de los componentes de un vector de tamaño n

```
1: function SUMA(A:vector): (return entero)
2:     m = 0;                                          ▷ 1 OE
3:     for (i = 1; i ≤ n; i++) do                      ▷ 2 OE por it
4:         m = m + A[i];                               ▷ 3 OE por it
5:     end for
6:     return (m);                                     ▷ 1 OE
7: end function
```

T(for) = T(Cond) + n_iter * (T(Cond)+T(Suma)+T(Bloque))

Ejemplo: suma de los componentes de un vector de tamaño n

```
1: function SUMA(A:vector): (return entero)
2:     m = 0;                                   ▷ 1 OE
3:     for (i = 1; i ≤ n; i++) do               ▷ 2 OE por it
4:         m = m + A[i];                        ▷ 3 OE por it
5:     end for
6:     return (m);                              ▷ 1 OE
7: end function
```



$$T(n) = 1 + 1 + 1 + n * (1+1+3) + 1 = 4 + 5 * n$$

T(for) = T(Cond) + n_iter*(T(Cond) + T(Suma) + T(Bloque)

Ejercicios

Supongamos que tenemos un vector **A** con **n** elementos, calcula el tiempo de ejecución de los siguientes algoritmos.

```
1: procedure ALGOR1(A:vector)
2:     j = 0;
3:     while (j < n) do
4:         A[j] = 2*j;
5:         j++;
6:     end while
7: end procedure
```

```
1: procedure ALGOR2(A:vector)
2:     for (i = 1; i ≤ n; i++) do
3:         for (j = 1; j ≤ n; j++) do
4:             A[j] = A[j]+i;
5:         end for
6:     end for
7: end procedure
```

Más ejercicios

Calcula el tiempo de ejecución de los siguientes algoritmos, cuidado con el cálculo del número de iteraciones del bucle.

```
1: procedure EJER1(n:entero)
2:     m = 0;
3:     for (i = 1; i ≤ n; i+=2) do
4:         m++;
5:     end for
6: end procedure
```

```
1: procedure EJER2(n:entero)
2:     m = 0;
3:     for (i = 1; i ≤ n; i*=2) do
4:         m++;
5:     end for
6: end procedure
```

SOLUCIÓN

El problema aparece cuando el número de iteraciones del bucle no coincide con el valor de n, y hay que estimarlo.

```
1: procedure EJER1(n:entero)
2:     m = 0;
3:     for (i = 1; i ≤ n; i+=2) do
4:         m++;
5:     end for
6: end procedure
```

i = 1, 3, 5, 7 ...

B = n/2 (núm. iteraciones)

coste for: 1 + B * (2+1)

coste total= 1 + 1 + coste for

```
1: procedure EJER2(n:entero)
2:     m = 0;
3:     for (i = 1; i ≤ n; i*=2) do
4:         m++;
5:     end for
6: end procedure
```

i = 1, 2, 4, 8, 16 ....
$i = 2^0, 2^1, 2^2, 2^3, 2^4$ ...

$2^B = n$ ➤ $B = \log(n)$

coste for: 1 + B * (2+1)

coste total= 1 + 1 + coste for

Más ejercicios

Calcula el tiempo de ejecución de los siguientes algoritmos.

```
1: procedure EJER3(n:entero)
2:     m = 0;
3:     for (i = n; i ≥ 1; i-=2) do
4:         m++;
5:     end for
6: end procedure
```

```
1: procedure EJER4(n:entero)
2:     m = 0;
3:     for (i = n; i ≥ 1; i/=2) do
4:         m++;
5:     end for
6: end procedure
```

Index

1. Objetivos
2. Noción de coste computacional
3. Cotas de complejidad
4. Análisis asintótico
5. Cálculo de complejidades

Cotas de complejidad

Dado un vector **X** de **n** números naturales y dado un número natural **z**

- encuentra el índice **i** tal que **X[i] = z**
- calcula el número de pasos que realiza

```
1: function BUSCA(X: vector, z:natural): (return natural)
2:     i = 0;
3:     while (X[i] ≠ z ∧ i ≤ n) do
4:         i = i + 1;
5:     end while
6:     if (i==n) then
7:         return (-1);                          ▷ no encontrado
8:     else
9:         return(i);                              ▷ encontrado
10:    end if
11: end function
```

Problema: no podemos contar el número de pasos de programa porque para diferentes entradas de un mismo tamaño de problema se obtienen diferentes complejidades [1]

X	z	**número pasos**
(1, 0, 2, 4)	3	$1 + 5 + 2 = 8$
(1, 0, 2, 4)	0	$1 + 2 + 2 = 5$
(1, 0, 2, 4)	1	$1 + 1 + 2 = 4$

¿Qué podemos hacer? acotar el coste mediante dos funciones que expresen respectivamente, el máximo y el mínimo coste del algoritmo (cotas de complejidad)

[1] supondremos que cada iteración del bucle while tiene coste 1

Tipos de análisis

Cuando el comportamiento de un algoritmo puede cambiar notablemente para diferentes entradas (p. e., lo ordenados que se encuentren ya los datos a ordenar) se introducen las cotas de complejidad.

Así suelen estudiarse básicamente dos casos para un mismo algoritmo: caso peor y caso mejor.

Caso mejor: traza del algoritmo que realiza menos instrucciones (**cota inferior** C_i)

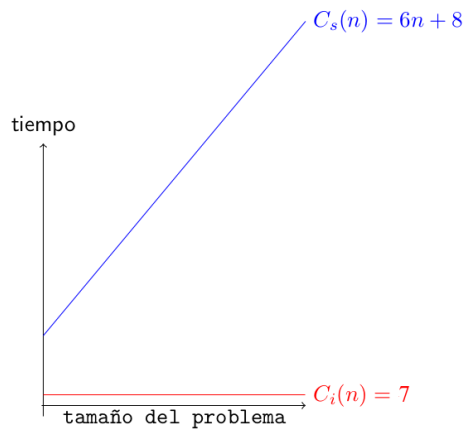
Caso peor: traza del algoritmo que realiza más instrucciones (**cota superior** C_s)

Ejemplo

```
1: function BUSCAR(X:vector, z:entero)(return entero)
2:     i = 0;
3:     while (X[i] ≠ z) ∧ (i < n) do
4:         i = i + 1;
5:     end while
6:     if (X[i]==z) then return(i);
7:     else return(-1);
8:     end if
9: end function
```

El tamaño del problema (n) es el número de elementos del vector X

Línea	Caso mejor	Caso peor
2	1	1
3,4	3	6n+4
6,7	3	3
coste	7	6n+8



Ejercicio con cotas: ordenación por intercambio

```
1:  procedure ORD1(A:vector)
2:      m = 0;
3:      for (i = 1; i < n; i++) do
4:          for (j = n; j > i; j- -) do
5:              if (A[j-1] > A[j]) then
6:                  x = A[j-1]; A[j-1] = A[j]; A[j] = x;
7:              end if
8:          end for
9:      end for
10: end procedure
```

¿Qué cotas tiene este algoritmo? Por simplificación, resuelve el problema cambiando la i por un 0 en la línea 4.

Ejercicio con cotas

El caso peor y mejor en este ejemplo se reducen a que se cumpla o no la condición del **if** en la línea 5.

```
1:  procedure ORD1(A:vector)
2:      m = 0;
3:      for (i = 1; i < n; i++) do
4:          for (j = n; j > i; j- -) do
5:              if (A[j-1] > A[j]) then
6:                  x = A[j-1]; A[j-1] = A[j]; A[j] = x;
7:              end if
8:          end for
9:      end for
10: end procedure
```

caso mejor = 4
caso peor = 4 + 9

Ejercicio con cotas: ordenación por inserción

El caso peor y mejor en este ejemplo se reducen a que se cumpla alguna de las condiciones del bucle **while** en la línea 5. Aquí la condición **(i>0)** sólo fallará al final, cuando se haya recorrido todo el vector.

```
1:  procedure ORD2(A:vector)
2:      for (j = 1; j < n; j++) do
3:          m = A[j];
4:          i = j;
5:          while (i>0) ∧ (A[i-1] > m) do
6:              A[i] = A[i-1];
7:              i = i - 1;
8:          end while
9:          A[i] = m;
10:     end for
11: end procedure
```

caso mejor: 5

caso peor: 2 + (n-1)*(5+6)

num. iteraciones

Index

1. Objetivos
2. Noción de coste computacional
3. Cotas de complejidad
4. Análisis asintótico
5. Cálculo de complejidades

Análisis asintótico

El estudio de la complejidad tiene sentido cuando hablamos de **tamaños grandes de problemas** debido a que:

- las diferencias "reales" en tiempo de ejecución de algoritmos con diferente coste para tamaños pequeños de problema no suelen ser significativos;
- es lógico invertir tiempo en el desarrollo de un buen algoritmo sólo si se prevé que éste realizará un gran volumen de operaciones;

Análisis asintótico

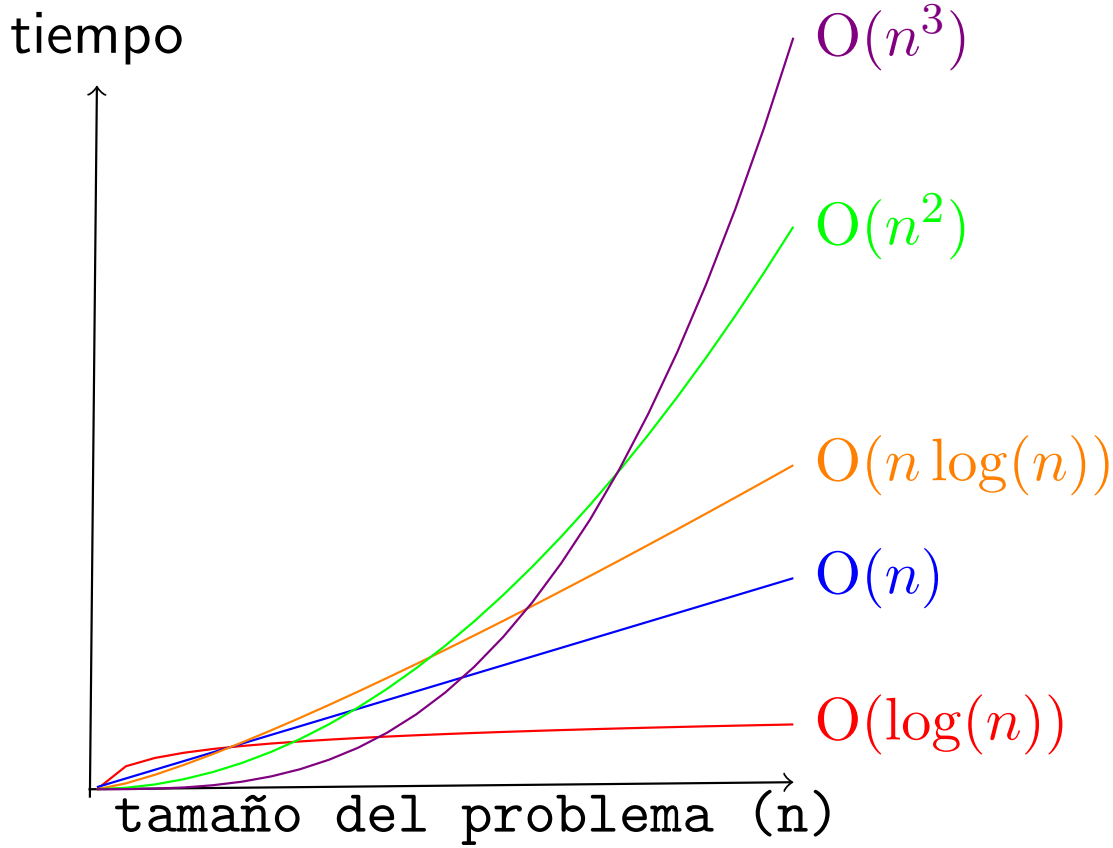
Al estudio de la complejidad para tamaños grandes de problemas ($n \to \infty$) se le denomina **análisis asintótico:**

- se definen **CLASES DE EQUIVALENCIA** que engloban a las funciones que "crecen de la misma forma";
- se emplea la **notación asintótica**; hay 3 tipos:

Cota superior	Notación O (big omicron)
Cota inferior	Notación Ω (omega)
Orden exacto	Notación Θ (big theta)

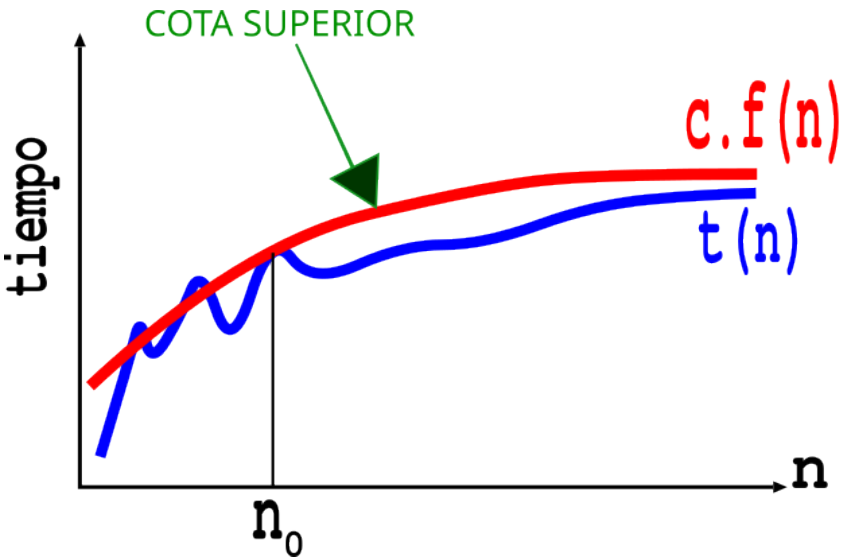
¿para qué sirven?

- nos permiten agrupar por **clases** las funciones con el mismo crecimiento
- las clases resultantes se representan con la función más simple



Cota superior. Notacion O

$t(n) \in \mathrm{O}(f(n))$ si a partir de un n_o se cumple que $t(n) \leq c \cdot f(n)$, con $c > 0$



Ejemplos de funciones $t(n)$:

1. $3n + 1 \in \mathrm{O}(n)$?
2. $3n^2 + 1 \in \mathrm{O}(n)$?
3. $3n^2 + 1 \in \mathrm{O}(n^2)$?

Haceos las preguntas:

- A $n \in \mathrm{O}(n)$?
- B $n \in \mathrm{O}(n^2)$?
- C $n \in \mathrm{O}(n^3)$?

¿Cuál es la mejor cota superior para n? (A, B o C) $\rightarrow$

Notación asintótica: algunas propiedades

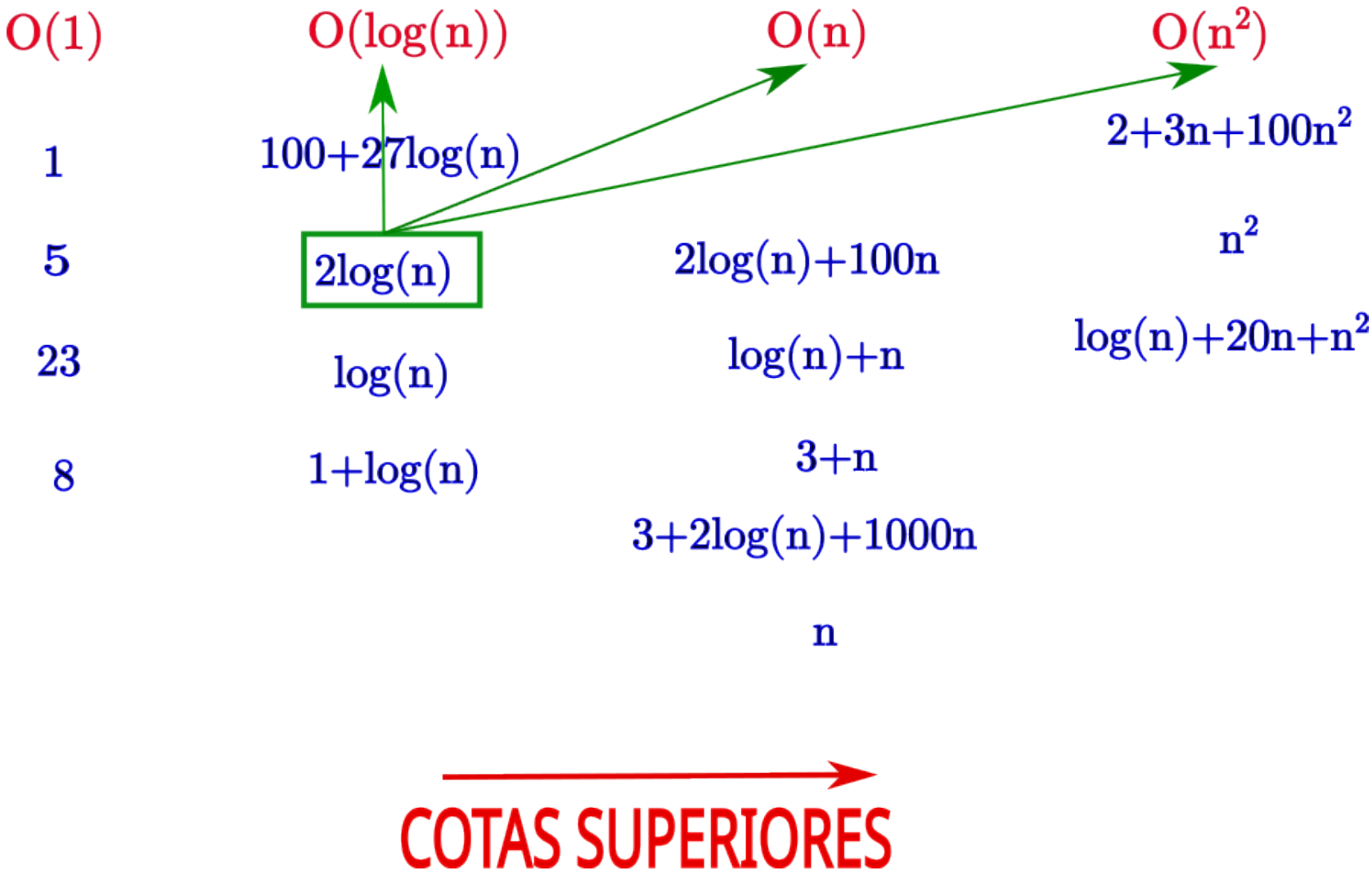
- **Regla de la suma:**
  $f_1 \in \mathrm{O}(g_1) \wedge f_2 \in \mathrm{O}(g_2) \rightarrow \boldsymbol{f_1 + f_2 \in \mathbf{O}(max(g_1, g_2))}$
  Ejemplo:
  $f_1 = n + 3; \qquad f_2 = n^2 - n + 1 \qquad \rightarrow \qquad f_1 + f_2 \in \mathrm{O}(n^2)$

- **Regla del producto:**
  $f_1 \in \mathrm{O}(g_1) \wedge f_2 \in \mathrm{O}(g_2) \rightarrow \boldsymbol{f_1 \cdot f_2 \in \mathbf{O}(g_1 \cdot g_2)}$
  Ejemplo:
  $\rightarrow \qquad f_1 \cdot f_2 \in \mathrm{O}(n^3)$

- $\boxed{f(n) = a_m \cdot \boldsymbol{n^m} + a_{m-1} \cdot n^{m-1} + \ldots + a_1 \cdot n + a_o \in \mathbf{O}(\boldsymbol{n^m})}$

Ejemplos de funciones

Cada función puede ser acotada superiormente por todas las funciones desde su columna hacia la derecha.



Complejidad asintótica de una función

Una función $f(n)$ puede estar acotada superiormente por un **número indefinido** de funciones a partir de ciertos valores n_0,

$$f(n) \leq g(n) \leq h(n) \leq k(n)...$$

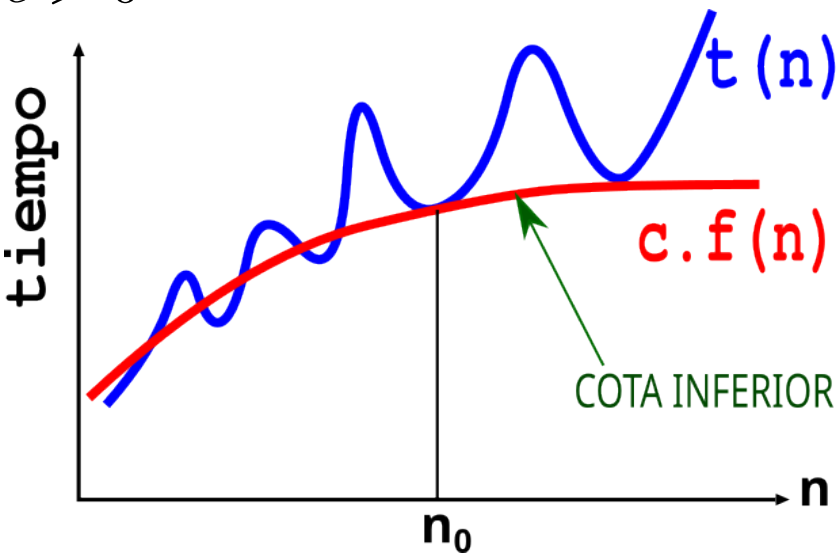
La **complejidad asintótica** de la función $f(n)$ se considera que es la cota superior **más ajustada**:

$$\boxed{f(n) = \mathrm{O}(g(n))}$$

Si un algoritmo es cuadrático, se dice entonces que su eficiencia es $\mathrm{O}(n^2)$. Esto significa que el tiempo de proceso del algoritmo crece con el cuadrado del número de entradas.

Otras notaciones. Cota inferior (Ω)

$t(n) \in \Omega(f(n))$ si a partir de un n_o se cumple que $t(n) \geq c \cdot f(n)$ con $c > 0$



Ejemplos de funciones $t(n)$:

1. $3n + 1 \in \Omega(n)$?
2. $3n^2 + 1 \in \Omega(n)$?
3. $3n^2 + 1 \in \Omega(n^2)$?

Haceos las preguntas:

A $n \in \Omega(n)$?

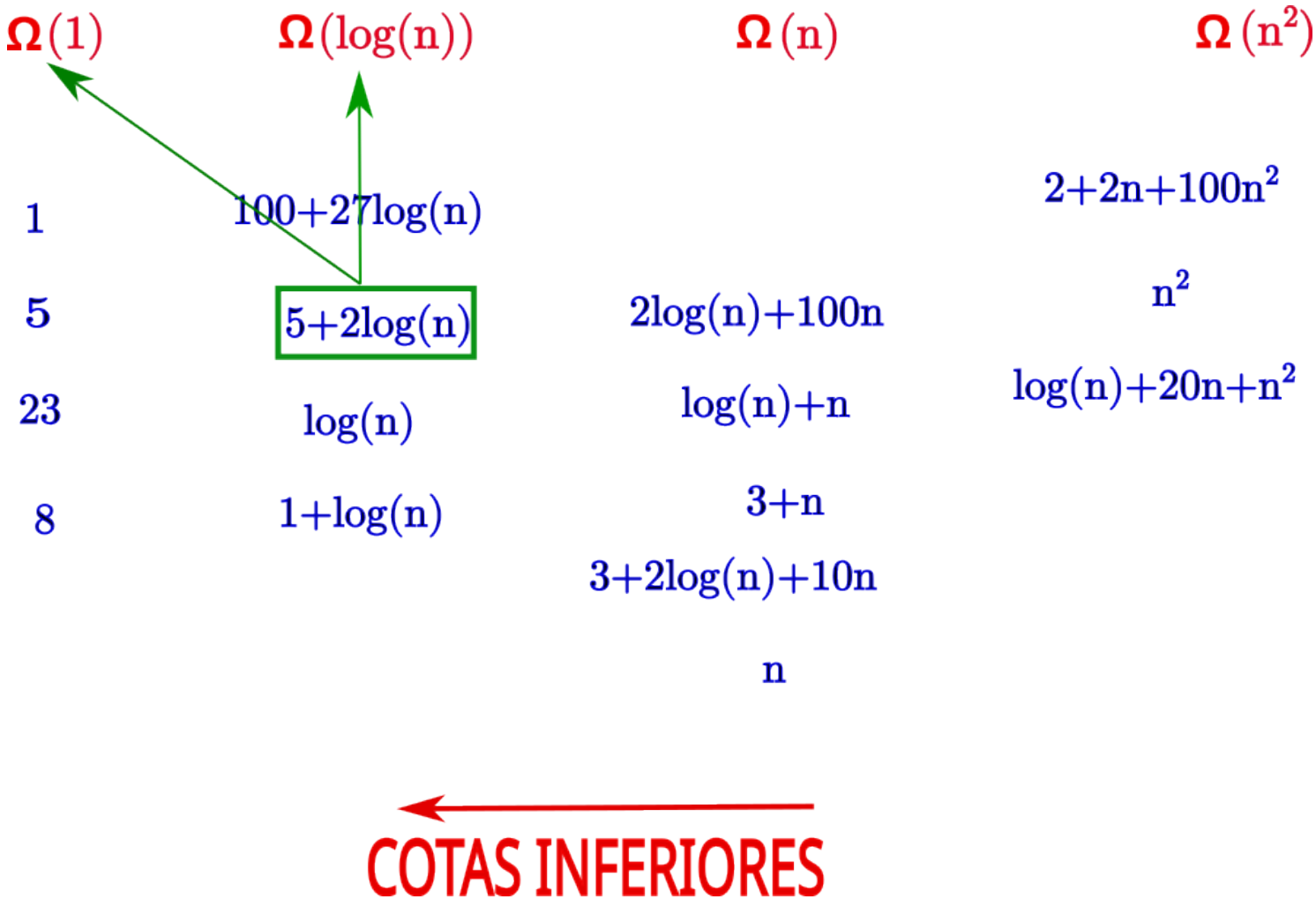
B $n \in \Omega(\log(n))$?

C $n \in \Omega(1)$?

¿Cuál es la mejor cota inferior para n? (A, B o C) $\rightarrow$

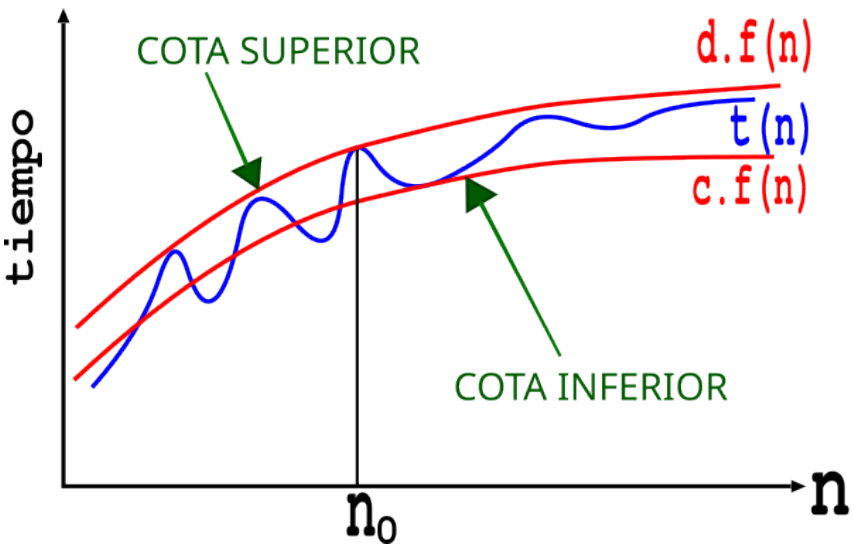
Ejemplos de funciones.

Cada función puede ser acotada inferiormente por todas las funciones desde su columna hacia la izquierda.
Cada conjunto incluye las funciones que acota inferiormente.



Otras notaciones. Orden exacto (Θ)

es equivalente a decir que $\Theta(f) = \mathrm{O(f)} \cap \Omega(\mathrm{f})$



Ejemplos de funciones $t(n)$:

1. $3n + 1 \in \Theta(n)$?
2. $3n^2 + 1 \in \Theta(n)$?
3. $3n^2 + 1 \in \Theta(n^2)$?

Jerarquía de funciones

Los conjuntos de funciones están incluídos unos en otros, generando una ordenación de las diferentes funciones. Así, las clases más utilizadas en la expresión de complejidades son:

$$\underset{\text{constante}}{\mathrm{O}(1)} \subset \underset{\text{sublog}}{\mathrm{O}(\log(\log n))} \subset \underset{\text{log}}{\mathrm{O}(\log n)} \subset \underset{\text{log}}{\mathrm{O}(\log^{a>1} n)} \subset$$

$$\underset{\text{sublineal}}{\mathrm{O}(\sqrt{n})} \subset \underset{\text{lineal}}{\mathrm{O}(n)} \subset \underset{\text{lineal-log}}{\mathrm{O}(n \log n)} \subset \underset{\text{polin}}{\mathrm{O}(n^2) \ldots} \subset$$

$$\underset{\text{polin}}{\mathrm{O}(n^{a>2})} \subset \underset{\text{exp}}{\mathrm{O}(2^n)} \subset \underset{\text{superexp}}{\mathrm{O}(n!)} \subset \underset{\text{superexp}}{\mathrm{O}(n^n)}$$

Jerarquía de funciones

eficiente	$\mathrm{O}(1)$	constante
	$\mathrm{O}(\log n)$	logarítmico
	$\mathrm{O}(n)$	lineal
	$\mathrm{O}(n \log n)$	casi lineal
tratable	$\mathrm{O}(n^2)$	cuadrático
	$\mathrm{O}(n^3)$	cúbico
	$\mathrm{O}(n^k)$	polinómico
intratable	$\mathrm{O}(2^n)$	exponencial
	$\mathrm{O}(n!)$	factorial

Clases de algoritmos y sus tiempos de ejecución

Supongamos que trabajamos en una computadora que realiza un millón de operaciones por segundo ($1s = 10^6\mu s = 10^3 ms$)

Clase	**Coste**	**Número de operaciones y tiempo de ejecución (1 instr/μs)**					
n		10		10^2		10^3	
constante	$O(1)$	1	1 μs	1	1 μs	1	1 μs
logarítmico	$O(\log n)$	3.32	3 μs	6.64	7 μs	9.97	10 μs
lineal	O(n)	10	10 μs	10^2	100 μs	10^3	1 ms
$O(n \log n)$	$O(n \log n)$	33.2	33 μs	664	664 μs	9970	10 ms
cuadrática	$O(n^2)$	10^2	100 μs	10^4	10 ms	10^6	1 s
cúbica	$O(n^3)$	10^3	1 ms	10^6	1 s	10^9	16.7 min
exponencial	$O(2^n)$	1024	10 ms	10^{30}	3×10^{17} años	10^{301}	
n		10^4		10^5		10^6	
constante	$O(1)$	1	1 μs	1	1 μs	1	1 μs
logarítmico	$O(\log n)$	13.3	13 μs	16.6	17 μs	19.93	20 μs
lineal	$O(n)$	10^4	10 ms	10^5	0.1 s	10^6	1 s
$O(n \log n)$	$O(n \log n)$	133×10^3	133 ms	166×10^4	1.6 s	199×10^5	20 s
cuadrática	$O(n^2)$	10^8	1.7 min	10^{10}	16.7 min	10^{12}	11.6 días
cúbica	$O(n^3)$	10^{12}	11.6 días	10^{15}	31.7 años	10^{18}	31709 años
exponencial	$O(2^n)$	10^{3010}		10^{30103}		10^{301030}	

Index

1 Objetivos

2 Noción de coste computacional

3 Cotas de complejidad

4 Análisis asintótico

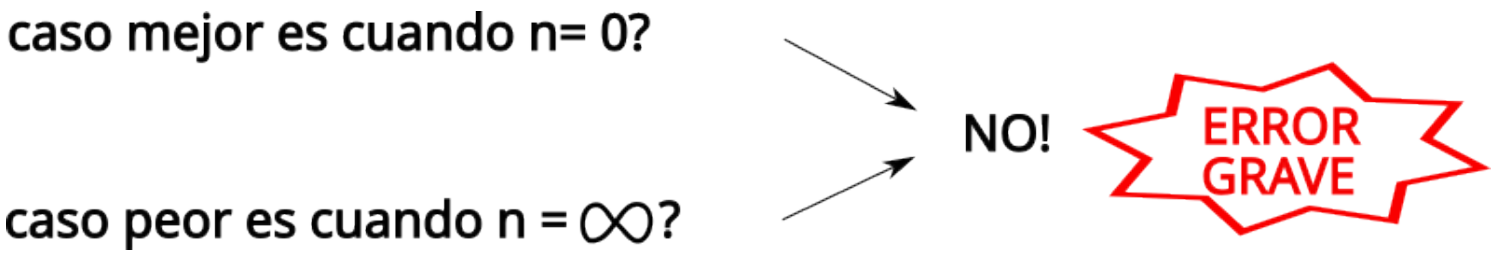
5 Cálculo de complejidades

Cálculo de complejidades

Etapas para obtener las cotas de complejidad

1. Determinar la **talla** o tamaño del problema
2. Determinar el **caso mejor y peor**: instancias para las que el algoritmo tarda más o menos
   - no siempre existe mejor y peor caso, ya que hay algoritmos que se comportan de igual forma para cualquier instancia del problema
3. Obtención de las cotas **para cada caso**
   - algoritmos iterativos
   - algoritmos recursivos

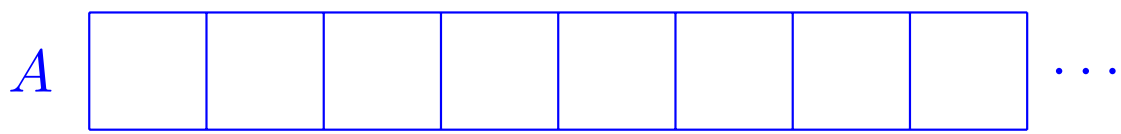
Un error demasiado frecuente



No hay un mejor y peor caso, sino un "mejor y peor caso para cada valor de n".

- El **mejor caso para cada valor de** n hace que el algoritmo se ejecute con el **menor número posible de pasos** (para cada valor de n).
- El **peor caso para cada valor de** n hace que el algoritmo se ejecute con el **mayor número posible de pasos** (para cada valor de n).

Ejemplo: búsqueda de un elemento en un vector A de tamaño genérico n



Algoritmos iterativos

```
1: function SUMAR(X: vector[1 ... n])(entero)
2:     s = 0;
3:     for (i = 1; i ≤ n; i++) do
4:         s = s + X[i];
5:     end for
6:     return(s);
7: end function
```

En este ejemplo $f(n)$ es única (no hay caso mejor y peor)

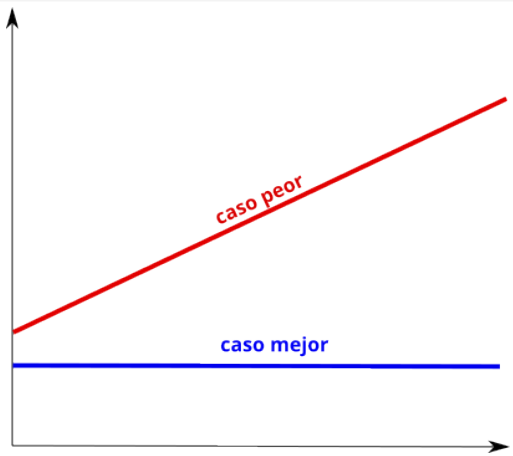
$f(n) = 5n + 4 \rightarrow f(n) \in \mathrm{O}(n)$ y $f(n) \in \Omega(n)$

$\Rightarrow f(n) \in \Theta(n)$

Algoritmos iterativos

```
1: function BUSCAR(X: VECTOR[1 ... n], z:ENTERO)(ENTERO)
2:     i = 0;
3:     while ((X[i] ≠ z) ∧ (i < n)) do
4:         i = i + 1;
5:     end while
6:     if (X[i] == z) then
7:         return(i)                                  ▷ ENCONTRADO
8:     else
9:         return(-1)                                 ▷ NO ENCONTRADO
10:    end if
11: end function
```

- **caso mejor:** $f(n) = 6$
- **caso peor:** $f(n) = 6n + 8$
- $\rightarrow f(n) \in \mathrm{O}(n)$ y $f(n) \in \Omega(1)$



Algoritmos recursivos

```
1: function F1(n: ENTERO)(ENTERO)
2:     if (n == 1) then return (1)
3:     else return (n·F1(n − 1))
4:     end if
5: end function
```

¿Cuál es la contribución al coste total del algoritmo de cada una de las llamadas recursivas?

Sólo se puede asegurar que el coste, f, será una función del tamaño del problema n, y su valor dependerá del coste de las sucesivas llamadas recursivas.

$$f(n) = \begin{cases} 1 & n \leq 1 \quad \text{CASO BASE} \\ 1 + f(n-1) & n > 1 \quad \text{CASO GENERAL} \end{cases}$$

El "caso base" **NO** es lo mismo que "el mejor caso".

Algoritmos recursivos

Una **relación de recurrencia** es una expresión que relaciona el valor de una función f definida para un entero n con uno o más valores de la misma función para valores menores que n.

$$f(n) = \begin{cases} P'(n) & n \leq n_0 \quad \text{CASO BASE} \\ a \cdot f(F(n)) + P(n) & n > n_0 \quad \text{CASO GENERAL} \end{cases}$$

donde

- $a \in \mathbb{N}$ es una constante
- $P(n), P'(n)$ son funciones de n
- $F(n) < n$ es una **función estrictamente decreciente** con n. Normalmente del tipo $(n - b)$ ó (n/b) con $b \in \mathbb{N}$.

Algoritmos recursivos

El coste de un algoritmo recursivo se obtiene en 2 pasos:

- obtener las ecuaciones de recurrencia del algoritmo
- resolver las ecuaciones **(método de sustitución)** que consiste en:
  1. sustituir cada $f(n)$ por su valor al aplicarle de nuevo la función, hasta obtener un término general en función de un número de iteraciones i
  2. utilizar el caso base para obtener i
  3. sustituir i en el término general

Un ejemplo

```
1: function MÁXIMO(X: VECTOR[ENTERO], n:ENTERO)(ENTERO)
2:     if (n == 1) then return(X[1]);
3:     else
4:         return(MAYOR(MÁXIMO(X, n − 1), X[n]));
5:     end if
6: end function
```

donde

- $\text{MAYOR} \in \Theta(1)$ (es constante)
- la talla del problema es n
- la ecuación de recurrencia a partir del algoritmo es

$$f(n) = \begin{cases} 1 & n = 1 \quad \text{CASO BASE} \\ 1 + f(n-1) & n > 1 \quad \text{CASO GENERAL} \end{cases}$$

Resolución por sustitución

Primer paso: sustitución

$$\begin{aligned} f(n) &= 1 + f(n-1) = && 1^a \text{recursión} \\ &= 1 + 1 + f(n-2) = && 2^a \text{recursión} \\ &= 1 + 1 + 1 + f(n-3) = \ldots && 3^a \text{recursión} \\ &= i + f(n-i) && \text{i-ésima recursión} \end{aligned}$$

Segundo paso: obtener ecuación para el número de iteraciones i, y obtener su valor

- $\boxed{\text{CASO BASE f(1)=1}} \rightarrow$ la recursión termina cuando $(n-i) = 1$
- despejando $i \rightarrow \boxed{i = n-1}$ **(número de llamadas recursivas)**

Tercer paso: sustituir i en la i-ésima recursión de $f(n)$

$$f(n) = i + f(n-i) = n - 1 + f(1) = n - 1 + 1 = n$$

$$\rightarrow f(n) \in \mathrm{O}(n)$$

Otro ejemplo

```
1: function F2(n:NATURAL)(NATURAL)
2:     if n ≤ 1 then return(1);
3:     else  return(n*F2(n/2))
4:     end if
5: end function
```

donde

- la talla del problema es n
- la ecuación de recurrencia a partir del algoritmo es

$$f(n) = \begin{cases} 1 & n \leq 1 \quad \text{CASO BASE} \\ 1 + f(\frac{n}{2}) & n > 1 \quad \text{CASO GENERAL} \end{cases}$$

¿qué sale desarrollando la ecuación de recurrencia?

ANEXO

Expresiones más frecuentes

$$1) c \in \Theta(1),\ \forall c \in \mathbb{R}^{\geq 0}$$

$$2) \sum_{i=1}^{n} i = \tfrac{n}{2}(n+1) \in \Theta(n^2)$$

$$3) \sum_{i=0}^{n} r^i = \tfrac{r^{n+1}-1}{r-1} \in \Theta(r^n), r \neq 1$$

Logaritmos

- $\log_a(n) = \frac{\log_b(n)}{\log_b(a)}$
- $\log_a(n \cdot m) = \log_a(n) + \log_a(m)$
- $\log_a(n^p) = p \log_a(n)$
- $n^{\log_a(m)} = m^{\log_a(n)}$